

SUPPLEMENTARY PROBLEM: GREEDY ALGORITHMS

Instructor: Orhan Özgüner

© Özgüner @CWRU, 2025

1 Problem 1

You are running the end-of-day processing for a financial institution. You have n different database tasks that have to be performed. Each task, t , consists of two parts; the first part is a database update that takes expected time w_t minutes, followed by post-processing that takes expected time p_t minutes. Only one task may update the database at a time, and tasks may not be interrupted once started. However, the post-processing can occur in parallel. You may not leave until all the tasks are complete. Given a list, T , of n tasks, devise an algorithm to output the fastest time it will take the tasks to complete, and the order of the tasks.

1.1 Example

Input: $T = [\{4, 3\}, \{2, 5\}, \{3, 6\}, \{1, 2\}, \{5, 4\}]$

Output:

- time = 17
- Order = $[t_3, t_2, t_4, t_5, t_1]$

2 Problem 2

One Halloween tradition is to hand out candy to kids dressed in costumes. You purchased a large bag of assorted candies to hand out. However, you did not purchase enough for the n kids who are coming to your house! You rush to the store to get more, but all they have left is candy corn that (almost) nobody likes. You get it anyway to make sure you have n pieces of candy, but now you have a problem. Who gets the good candy and who gets the candy corn?

Specifically, you have n pieces of candy and n kids visiting. The candy is ranked so candy i has value v_i where $v_i > v_j$ means candy i is preferred by all children to candy j (lucky for you, all kids in your neighborhood rank the candy the same). However, some children are more picky than others. Each child k has a preference p_k such that if they receive a candy with a value greater or equal to p_k they will be happy, but anything less and they will be upset.

Each child has a parent with weight, w_k , representing how much you like the parent. If $w_a > w_b$, then you like the child a 's parent more than you like the child b 's parent. Your goal is to give out candy in order to not upset too many of the children whose parents you like. Specifically, devise an algorithm to determine which child gets which piece of candy so that you **minimize the sum of the weights of the upset childrens' parents**.

2.1 Example

Input:

1. $n = 5$
2. Candy values $v = 3, 1, 5, 2, 4$
3. Preferences $p = 2, 3, 5, 4, 3$
4. Weights $w = 7, 3, 5, 10, 2$

Output:

1. Total upset weight = 2

3 Problem 1 Solution

Greedy Choice: There is an optimal ordering such that the tasks are kicked off in decreasing order of post-processing time, i.e. $\exists O = \{t_1, \dots, t_n\}$ s.t. $p_1 \geq \dots \geq p_n$.

3.1 Proof of Correctness

Let's assume that there is no idle database update time during the execution of tasks, i.e. once task i has finished its database update, then task $i + 1$ will start its database update right away. The same assumption can be made about the post-processing of a task after its database update, i.e. it must start right away. These assumptions assure that no time is wasted in between running tasks.

Let S be an optimal ordering of tasks. If S is in decreasing order of post-processing time, then we are done. Otherwise, there must be an instance where $p_i < p_{i+1}$. Consider task i starting its database update at time m_i . Therefore, the ending time of task $i + 1$ can be represented as

$$\begin{aligned} p_i < p_{i+1} &\implies w_i + p_i < w_i + p_{i+1} \\ \implies m_{i+1} &= m_i + \max\{w_i + p_i, w_i + w_{i+1} + p_{i+1}\} \\ &= m_i + w_i + w_{i+1} + p_{i+1} \end{aligned}$$

The two situations represent either task i finishing last, or task $i + 1$ finishing last. Based on the assumption that $p_i < p_{i+1}$, we should that task i must always finish last with the timestamp given above. But what if we were to substitute our greedy choice into solution S' ? That is, if we swap the order of tasks i and $i + 1$ to create solution S' then we observe the following

$$\begin{aligned} p_i < p_{i+1} &\implies w_i + p_i < w_i + p_{i+1} \\ \implies m'_{i+1} &= m_i + \max\{w_{i+1} + p_{i+1}, w_{i+1} + w_i + p_i\} \\ \implies m_{i+1} &\geq w_{i+1} + p_{i+1} \\ &\text{and} \\ m_{i+1} &\geq w_{i+1} + w_i + p_i \\ \implies m_{i+1} &\geq m'_{i+1} \end{aligned}$$

Since we assumed that S was an optimal ordering, and showed that the ending time of S' must be less than or equal to S , then there must always exist an optimal solution with our greedy choice.

3.2 Pseudocode

Algorithm 1 Task Ordering Problem

```
1: procedure FASTEST-COMPLETION-TIME( $T, W, P, n$ )
2:   sort  $T$  in decreasing order with key =  $P$  ensuring originally corresponding elements from
    $W$  and  $P$  stay together
3:   update-time  $\leftarrow \sum_{i=1}^n W[i]$ 
4:   finish  $\leftarrow \infty$ 
5:   for  $i \leftarrow n$  to 1 do
6:     finish  $\leftarrow \min\{\text{finish}, \text{update-time} + P[i]\}$ 
7:     update-time  $\leftarrow \text{update-time} - W[i]$ 
8:   end for
9:   return finish,  $T$ 
10: end procedure
```

3.3 Runtime Analysis

Assuming that T is sorted efficiently, then line 2 of the pseudocode takes $n \log n$ time to complete. Calculating the total update time on line 3 takes n work to complete. Finally, the loop completes n iterations, with each iteration having a constant number of operations. Therefore, the total runtime of the algorithm is

$$T(n) = n \log n + n + n = \Theta(n \log n)$$

4 Problem 2 Solution

Greedy Choice: Assign each child the minimum candy value that satisfies their preference p_i if possible. If a child cannot be satisfied (i.e., if $v_i \not\geq p_i$ for the candy they receive), give them the lowest candy value remaining.

4.1 Proof of Correctness

Assume there exists an optimal solution S that minimizes the sum of the weights of the upset children's parents. If S follows our greedy choice, then we are done. Otherwise, there must be a point where S diverges from our greedy solution, resulting in a pair of children i and $i + 1$ such that one of the following conditions holds:

1. $w_i > w_{i+1}$ but S assigns a lower-value candy to child i than to child $i + 1$ (i.e., $v_i < v_{i+1}$), even though this lower-value candy does not satisfy child i 's preference (i.e., $v_i < p_i$).
2. $w_{i+1} > w_i$, meaning child $i + 1$'s parent is more highly weighted, but in S , child $i + 1$ receives a candy that does not meet their preference, while child i receives a candy that does.

For each case, we can show that rearranging the assignments to match our greedy strategy does not increase the total upset weight and may possibly even reduce it.

Case 1:

Suppose $w_i > w_{i+1}$ and $v_i < v_{i+1}$ in S , with $v_i < p_i$. In this case, child i is upset because they did not receive a candy that meets their preference. By swapping the assignments so that child i receives v_{i+1} (the higher value candy), we ensure that child i is now satisfied.

In the new solution S' :

1. If child $i + 1$ remains satisfied with the lower-value candy v_i , then the total upset weight decreases because we have satisfied child i , whose parent has a higher weight.
2. If child $i + 1$ becomes upset, the total upset weight remains the same or decreases because we prioritized satisfying the child with the higher-weighted parent (child i).

Thus, S' is at least as optimal as S and may be better, contradicting the assumption that S was an optimal solution with a lower upset weight than the greedy solution.

Case 2:

Suppose $w_{i+1} > w_i$ and S assigns a candy to child $i + 1$ that does not satisfy their preference (i.e., $v_{i+1} < p_{i+1}$), while child i is satisfied. By swapping the assignments so that child $i + 1$ receives a higher-value candy, we prioritize satisfying the more important parent. In this new solution S' , either:

1. Child $i + 1$ becomes satisfied and child i remains satisfied, reducing the total upset weight,
2. Only child i is now upset, but the total upset weight does not increase because we have reduced the upset weight for the higher-weighted parent.

By applying this argument to all remaining inversions (where an inversion is defined as an instance where $w_{i+1} > w_i$), we can transform S into an optimal solution that follows the greedy choice without increasing the total upset weight.

Since the greedy choice does not increase and may decrease the total upset weight, our greedy approach is optimal.

4.2 Pseudocode

Algorithm 2 AssignCandy

```

1: procedure ASSIGNCANDY( $n, v_i, p_i, w_i$ )
2:   Sort the children by their parent weights  $w_i$  in descending order and children respectively
3:   Sort the candies by value in ascending order
4:   Initialize an empty list assignments
5:   for  $i$  in range 1 to  $n$  do
6:      $j \leftarrow n$ 
7:     while  $v_j < p_i$  do
8:        $j \leftarrow j - 1$ 
9:     end while
10:    assignments[ $i$ ]  $\leftarrow$  candy[ $j$ ]
11:     $v_j \leftarrow 0$ 
12:  end for
13:  return assignments
14: end procedure

```

4.3 Runtime Analysis

Sorting the children and candies each takes $O(n \log n)$ time. The loop on line 5 involves checking each child against up to n candies, leading to a best-case runtime of $O(n \log n)$ if each check is quick, and a worst-case runtime of $O(n^2)$ if each candy requires multiple comparisons.

$$T(n) = O(n \log n) + O(n^2) = O(n^2)$$